GPU Instancing: Implementation and Results Summary

Project: Stanford Bunny rendering with C++ and OpenGL | Report date: 25 October 2026

1. Objective

Lab 02 introduces GPU instancing as an improvement over the individual draw-call approach studied in Lab 01. The objective is to reuse a single Stanford Bunny mesh and render many instances while reducing CPU-side draw-command submission overhead.

2. Rendering method

Lab 01: The CPU submits separate OpenGL draw calls for objects. Lab 02: The CPU submits an instanced draw call, allowing the GPU to process many copies of shared geometry with instance-specific transformations. The CPU still issues the OpenGL command; instancing does not make the GPU independently issue draw calls.

The Stanford Bunny mesh used in the project contains 2,503 vertices and 4,968 triangles. Instancing reuses the mesh rather than duplicating its vertex data for every Bunny. Each instance can have its own model transformation.

Conceptual OpenGL call: **glDrawElementsInstanced(GL_TRIANGLES, indexCount, GL_UNSIGNED_INT, nullptr, instanceCount)**. The exact Lab 02 source implementation was not provided for verification in this report.

3. Educational outcome

• Demonstrates the distinction between CPU draw-call submission and GPU vertex/fragment processing.

• Shows how one mesh can be reused for many objects with different transforms.

• Establishes the instancing foundation later extended in Lab 03 with an adaptive grid and automatic Bunny scaling.

• Instancing reduces submission overhead; it does not automatically reduce triangles per instance.

4. Quantitative evidence: Lab 03 continuation (not standalone Lab 02 results)

The following readings were captured in Lab 03_BunnyGrid_Adaptive, which continues the instanced rendering approach of Lab 02. They should not be presented as isolated Lab 02 performance measurements or as a before/after comparison against Lab 01.

Bunnies	Triangles / frame	AMD Radeon FPS	RTX 4050 FPS
10	49,680	4,098.1	2,718.4
100	496,800	1,159.9	2,423.3
1000	4,968,000	241.1	589.4
10000	49,680,000	55.5	64.2

At 10,000 Bunnies, the application submits 49.68 million triangles per frame. The screenshots show 55.5 FPS on the integrated AMD Radeon and 64.2 FPS on the NVIDIA RTX 4050 (approximately 1.16x RTX advantage). These are individual on-screen readings in an x64-Debug configuration, not averaged benchmark runs.

5. Interpretation and limitations

The Lab 03 results demonstrate that the instanced rendering pipeline was exercised on both GPUs, but they do not quantify how much faster instancing is than Lab 01. No matched non-instanced measurements were supplied. FPS alone cannot identify whether the remaining bottleneck is CPU submission, vertex processing, shading, memory, or other work.

6. Recommended follow-up experiment

For a controlled Lab 01 versus Lab 02 comparison, run the same Bunny counts, camera, viewport, shaders, and hardware with traditional draw calls and instanced drawing. Use x64-Release, disable VSync, warm up the scene, record repeated frame-time measurements, and verify the active GPU with GL_RENDERER. Then compare CPU and GPU timings separately.

Conclusion

Lab 02 established GPU instancing as the rendering strategy for multiple Stanford Bunnies. Lab 03 successfully extends that strategy to a dynamically sized adaptive grid and provides preliminary integrated-versus-dedicated GPU performance observations. A separate matched benchmark is still needed to measure the speedup attributable specifically to instancing.